

A Preregistered Paired Evaluation of Generate–Verify–Repair (GVR) for Intent→Configuration NetOps: Null Results on a Synthetic Verifier-Checked Task Bank

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, archived paired experiment testing whether a deterministic verifier plus a bounded LLM-guided repair loop (Generate–Verify–Repair, GVR) reduces verifier-detected semantic invariant violations for intent→device-configuration translation. The confirmatory design used a public synthetic NetOps task bank ($N = 240$ paired tasks), a single generator/prompt pairing (requested model: gpt-5-mini), and a pybatfish-style deterministic validator (deterministic_netops_validator_v5) running the full_intent_constraint_validation rule. Execution used shared-initial-candidate semantics (one generated candidate per task reused across arms) and allowed at most one bounded repair attempt per task. All planned episodes completed (planned_episode_count = 480; completed_episode_count = 480) with model_calls_used = 240 (shared-generation). Every generated candidate satisfied the deterministic verifier in both arms: $n_{11} = 240$, $n_{10} = n_{01} = n_{00} = 0$. The paired estimate (rate_GVR - rate_baseline) = 0.0; exact McNemar two-sided $p = 1.00$; paired-bootstrap 95% CI = [0.0, 0.0] (10,000 resamples, bootstrap_seed = 1729). Two preregistration/execution issues affect interpretation: (1) shared-generation semantics removed the per-arm stochasticity assumed by the preregistered power calculation, and (2) the prespecified schema-constrained-baseline comparison was not executed. Conservatively interpreted, bounded GVR provided no additional verifier-detectable benefit in this executed artifact-backed setting because no verifier-detectable errors occurred; results do not generalize beyond the executed semantics, model, task bank, repair budget, or verifier.

I. INTRODUCTION

LLM-driven intent→configuration translation promises operational efficiency for network operators but raises safety concerns: stochastic generators can emit syntactically plausible yet semantically incorrect commands that violate network invariants (reachability, required workflow sequences, protected interfaces). Prior work therefore recommends interposing deterministic validators and iterative repair (generate–verify–repair, GVR) so mechanically checkable defects are detected and corrected before execution. We preregistered and executed a confirmatory paired experiment (study c1_gvr_semantic_eval_prereg_v1) to test whether bounded GVR (a deterministic verifier plus at most one bounded LLM repair attempt) reduces verifier-detected semantic invariant violations relative to an unconstrained LLM baseline (and, prespecified, relative to a schema-constrained baseline). The

study used a synthetic NetOps task bank of 240 paired tasks and a single generator/prompt pairing (requested model gpt-5-mini). The deterministic verifier implemented topology-aware pybatfish-style checks that return structured diagnostics (violation_count and explicit violation lists). The preregistration, analysis executor, and execution manifest were frozen prior to observing results and form the authoritative record for this run.

II. RELATED WORK

Three literature streams motivate our study. First, empirical surveys and experiments document LLM hallucinations and incorrect outputs for code and structured artifacts, motivating deterministic verification in pipelines; these studies demonstrate that unconstrained generation may produce plausible but incorrect artifacts that fail deeper invariants. Second, methodological work on GVR and constrained decoding reports improvements in mechanically checkable properties (compilation, syntactic validity, schema compliance) when deterministic validators or grammar constraints are interposed between generation and acceptance. These case studies motivate GVR but also emphasize that gains on syntactic or executable metrics do not automatically imply domain-level semantic correctness for complex NetOps invariants. Third, NetOps-specific research advocates verifier-driven evaluation (reachability and policy invariants) and intent-based translation systems that integrate domain semantics and workflow policies; prototype systems commonly combine LLMs with validators and recommend human oversight for high-stakes changes. Our run differs by being preregistered with a frozen analysis executor, using a paired confirmatory design on a synthetic task bank with explicit deterministic verifier predicates, and by executing under shared-initial-candidate semantics that removed within-pair stochastic variation.

III. METHODOLOGY

Preregistered estimand and hypotheses. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline computed by a frozen analysis executor; the preregistration specified Bonferroni correction across two primary

comparisons and a fixed-N design ($N = 240$ pairs). Study implementation. The experiment used adapter_family hosted_netops_gvr_v1 and requested generator gpt-5-mini. Critical executed parameters were generation_semantics = shared_initial_candidate (independent_condition_generation = false), initial_generation_calls_per_task = 1, and maximum_repair_calls_per_task = 1 (single bounded repair). The deterministic validator applied a full_intent_constraint_validation scoring rule that checks required command sequences, topology-aware reachability, protected-interface preservation, and final-state predicates. Task bank. The synthetic task bank encodes initial device states, natural-language intents, workflow constraints (required_sequence), and expected final states. Tasks include difficulty metadata (changed_interface_count, dependency_rule_count, and required_command_count) that the validator uses when producing structured validation traces. Analysis plan. The preregistered analysis executor performed an exact McNemar two-sided test on paired outcomes and computed absolute risk reductions with paired-bootstrap percentile confidence intervals (bootstrap_resamples = 10,000, bootstrap_seed = 1729). Missingness rules and conservative sensitivity analyses were preregistered; the executed run produced no failed episodes or flagged contamination items. Pre-specified comparisons. The design included two primary contrasts (GVR vs unconstrained baseline and GVR vs schema-constrained decoding); the latter comparison was prespecified but was not executed in the archived run and therefore is absent from the confirmatory results.

IV. RESULTS

Execution accounting and primary outcomes. Planned_episode_count = 480 (two conditions $\times$ 240 tasks); completed_episode_count = 480; failed_episode_count = 0; complete_pair_count = 240; model_calls_used = 240 (shared-initial-candidate semantics produced one initial generation per task reused across arms). Every shared initial candidate satisfied the deterministic verifier for every task in both arms: contingency counts $n_{11} = 240$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$. Baseline_success_rate = 1.0; guarded_success_rate = 1.0; paired estimate (rate_GVR - rate_baseline) = 0.0. Exact McNemar two-sided $p = 1.00$. Paired-bootstrap 95% CI (percentile; 10,000 resamples, seed = 1729) = [0.0, 0.0]. Repair usage. For every episode the validator trace records repair_applied = false; the bounded repair loop never executed because the shared initial candidate satisfied all verifier predicates across the task set. Interpretation of degeneracy. The absence of within-pair discordances (zero discordant pairs) makes the paired comparison degenerate: there is no within-pair information to estimate a differential effect. This outcome follows directly from the executed shared-generation semantics, which violate the preregistered independent-generation assumption and deterministically reduce discordance potential.

V. DISCUSSION

Conservative scientific interpretation. In the executed setting (single generator/prompt pairing, shared-initial-candidate semantics, synthetic task bank, single bounded repair attempt, deterministic verifier), bounded GVR produced no verifier-detectable benefit because there were no verifier-detectable errors to correct. This does not imply GVR is ineffective generally—measurable GVR benefit depends on generator stochasticity, verifier coverage, repair budget, and task difficulty. Design lessons. Two evaluation design choices determine ability to detect GVR benefits: (1) generation semantics (shared vs independent) set the discordance potential between arms—if the goal is to measure GVR’s efficacy correcting stochastic generator errors, independent per-condition generation is required; (2) verifier completeness and adversarial task selection determine whether the task set produces verifier-detectable failures. The preregistration planned golden-reference injections and adversarial checks to validate verifier sensitivity; the archived adapter-configured run did not realize the golden-injection step. Operational recommendations. To restore the preregistered inferential assumptions and robustly evaluate GVR we recommend: (a) rerunning the frozen task bank with independent_condition_generation = true (the run’s manifests include deterministic seeds to enable reproducible reruns under independent generation); (b) injecting adversarial/golden tasks (the preregistration specified a 10% golden task plan) to validate verifier sensitivity; (c) expanding generator scope to multi-model runs and increasing the repair budget to measure iterative repair effects; and (d) broadening verifier coverage (additional semantic invariants or formal hybrids) to reduce oracle incompleteness risk. Threats to validity. Internal validity: the executed shared-generation semantics violated the preregistered independent-generation power assumption and removed discordance potential. Construct validity: passing the deterministic verifier does not guarantee absence of operational risks beyond the verifier’s encoded invariants. External validity: synthetic tasks, one model (gpt-5-mini), and a single repair budget limit generalization. Conclusion validity: zero discordant outcomes preclude evidence that GVR changes verifier-detectable rates under the executed semantics.

VI. LIMITATIONS

- Synthetic task bank and verifier scope: $N = 240$ tasks were synthetic and evaluated against a pybatfish-style deterministic validator; results may not generalize to production topologies, vendor-specific syntax, or semantic invariants not encoded in the verifier.
- Shared-generation semantics: the archived run used generation_semantics = shared_initial_candidate (one initial generation per task reused across arms). This design reduced model-call cost but also removed per-arm stochastic variation and therefore reduced sensitivity to detect GVR benefits that rely on independent per-condition generator variability; the preregistered power calculation

assumed independent per-condition generation and therefore does not apply to the executed design.

- Single-model evaluation: the experiment used one generator (gpt-5-mini); cross-model generality is not established.
- Repair budget: only a single bounded repair iteration was permitted; different or larger repair budgets or iterative strategies might produce different outcomes.
- Verifier completeness risk: passing the deterministic verifier does not guarantee absence of all operational risks (unobserved invariants or properties outside the verifier model).
- Prespecified comparison not executed: the preregistration included a prespecified schema-constrained-baseline comparison that was not executed in the archived run; the artifacts do not record a rationale, which is an unresolved procedural limitation.

VII. CONCLUSION

We executed a fully preregistered, artifact-archived paired experiment evaluating bounded GVR for intent→configuration NetOps on a public synthetic task bank. Under the executed shared-generation semantics every generated candidate satisfied the deterministic verifier ($n11 = 240$), yielding a degenerate null paired estimate (0.0), exact McNemar $p = 1.00$, and paired-bootstrap 95% CI = [0.0, 0.0]. Conservatively interpreted, bounded GVR did not add verifier-detectable value in this artifact-backed setting because no verifier-detectable errors occurred. Two preregistration/execution deviations constrain inference: (1) shared-generation semantics invalidated the preregistered independent-generation power assumption; and (2) the prespecified schema-constrained-baseline comparison was not executed and the archived artifacts contain no recorded rationale. We release the authoritative archived artifacts so others can reproduce the accounting and extend the evaluation under independent-generation, adversarial/golden-task injections, multi-model regimes, and expanded verifier coverage to more robustly assess GVR's operational value. Evidence-to-claim mapping (concise): (i) `planned_episode_count = 480`, `completed_episode_count = 480`, `model_calls_used = 240` (shared-generation) — recorded in the execution manifest; (ii) every generated candidate passed the deterministic verifier in both arms ($n11 = 240$; $n10 = n01 = n00 = 0$) — recorded in the analysis contingency and validator traces; (iii) no repairs were applied (`repair_applied = false` for all episodes); and (iv) paired estimate = 0.0 with exact McNemar $p = 1.00$ and paired-bootstrap 95% CI = [0.0, 0.0] (10,000 resamples, seed = 1729). These specific numerical claims are reported from the frozen analysis executor and execution accounting produced by the archived run. Representative example. A typical task's reference and generated configuration used in the run is: "interface edge1 admin down; interface edge1 vlan 30; interface edge1 mtu 1600; interface edge1 admin up"; validator traces for that task record `violation_count = 0` before and after validation and `repair_applied = false`. Overall implication. The artifact-backed

null outcome emphasizes that evaluation semantics and verifier coverage determine whether GVR benefits are measurable; future evaluations should restore per-arm stochasticity and include adversarial/golden-task injections to test GVR under discordant-generation regimes where repairs are needed and measurable differences can arise.

DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). The immutable initial master prompt for the autonomous run is archived and available as `provenance/master_prompt.txt` (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human Research Director selected the topic family pre-lock: Generative AI and Large Language Models for NetOps with emphasis on reliability and verifier-assisted generation. Post-lock autonomous actions performed by the agentic research pipeline included literature discovery, preregistration freeze, experiment design, execution, analysis, and manuscript drafting. Models and components used in the executed run were the requested generator gpt-5-mini, an adapter/orchestration family implementing the `hosted_netops_gvr_v1` experiment semantics, and a deterministic validator implementation (`deterministic_netops_validator_v5`) applying the `full_intent_constraint_validation` rule. Executed numeric parameters recorded in the archived run were: `planned_episode_count = 480`; `completed_episode_count = 480`; `model_calls_used = 240` (shared-initial-candidate semantics produced one initial generation per task reused for both scored conditions); and `repair_budget = one` bounded repair attempt per task. The primary estimand (`paired_success_rate_difference_guarded_minus_baseline`) and the analysis executor (`paired_binary_analysis_v1`) were preregistered and frozen prior to observing outcomes. The archived execution and analysis artifacts contain the authoritative records for all reported numerical claims. The prespecified schema-constrained-baseline comparison was not executed in the archived run; the execution artifacts contain no recorded rationale for that omission. No manual edits to scientific content, figures, tables, references, or arguments occurred after the autonomy lock.

REFERENCES

- [1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [2] Sakshyam Banjade, "Schema-Constrained LLM Planning for Executable Molecular Workflows: An Intent-to-Execution Infrastructure for Chem-informatics", 2026, 10.26434/chemrxiv.15005091/v1
- [3] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [4] Hanli Peng, Yongsun Zheng, Ziyao Liu, Kwok-Yan Lam, "Tool Execution Hallucination in LLM-based Agents: A Unified Taxonomy with Detection, Mitigation, and Future Directions", 2026, 10.36227/techrxiv.177219979.94060974/v1

- [5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537
- [6] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567
- [7] Zhiyuan Wang, Jinhao Duan, Lu Cheng, Yue Zhang, Qingni Wang, Xiaoshuang Shi, Kaidi Xu, Heng Tao Shen, Xiaofeng Zhu, "ConU: Conformal Uncertainty in Large Language Models with Correctness Coverage Guarantees", 2024, 10.18653/v1/2024.findings-emnlp.404
- [8] Dmitriy Fedrushkov, Yulong He, Ivan Smirnov, Artem Aliev, Sergey Kovalchuk, "Semantic vs. Structural Signals: Log-Probability and LLM-as-a-Judge for Reference-Free Code Evaluation", 2026, 10.18653/v1/2026.gem-main.55
- [9] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, Zibin Zheng, "LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation", 2025, 10.1145/3728894